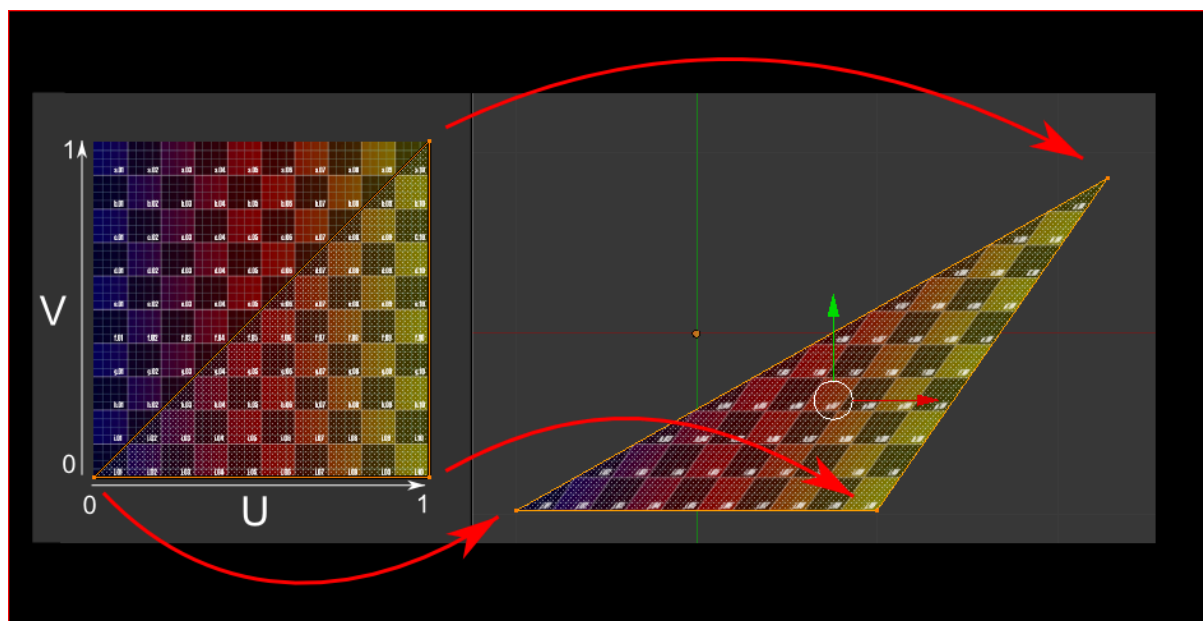
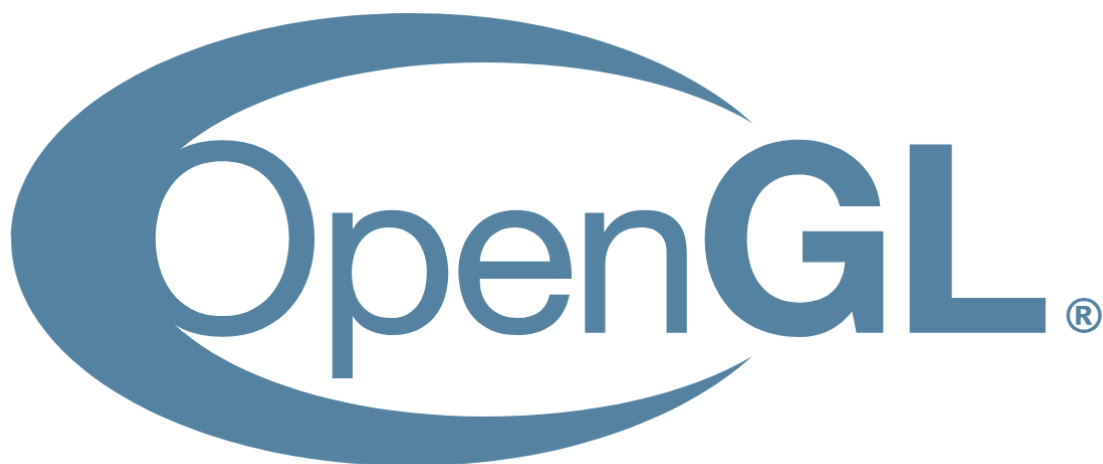


Entrega Final Open GL



Índice

Arquitectura general.....	4
Control de cámara.....	6
Fundamentos matemáticos.....	6
Sistema de control.....	8
Representación espacial.....	10
Integración en la escena.....	10
Mallas de elevación.....	11
Principio de funcionamiento.....	11
Generación de la geometría.....	12
Optimización mediante indexación.....	13
Mejora del realismo.....	14
Integración en el sistema de renderizado.....	14
Grafo de escena.....	16
Estructura del sistema.....	16
Transformaciones jerárquicas.....	16
Recorrido y renderizado.....	16
Ventajas del enfoque.....	17
Integración en la escena.....	17
Scene.....	17
Estructura de la escena.....	19
1. Terreno (Terrain).....	19
2. Cubo principal.....	19
3. Cubo secundario (animado).....	20
Texturas/transparencias.....	28
Texturas.....	28
Transparencia.....	28
Skybox/postproceso.....	31
Optimización.....	34
Conclusiones.....	36

Introducción

El presente proyecto consiste en el desarrollo de una escena tridimensional interactiva utilizando **OpenGL y C++**, aplicando técnicas avanzadas de programación gráfica moderna. El objetivo principal es construir un entorno 3D completo que integre sistemas de renderizado, gestión de cámaras, generación de terreno, iluminación y organización de escena mediante estructuras jerárquicas.

Para ello, se ha diseñado una arquitectura modular basada en programación orientada a objetos, que permite separar claramente las distintas responsabilidades del sistema (gestión de recursos, renderizado, control de cámara y lógica de escena). Esta estructura facilita la escalabilidad del proyecto y su mantenimiento, además de aproximarle al funcionamiento de un motor gráfico real.

El proyecto incluye la implementación de elementos fundamentales de la computación gráfica como el uso de shaders GLSL, buffers de vértices (VBO/EBO), texturizado, skybox, iluminación basada en modelos de Phong y generación procedural de terreno mediante heightmaps.

En conjunto, el sistema desarrollado permite la visualización e interacción en tiempo real de una escena 3D compleja, optimizada para su ejecución eficiente en GPU.

<https://github.com/avillarroya24/Practica-Final>

Lo he hecho todo en un repositorio de GitHub para poder ir subiendo todos los cambios necesarios durante todo el proyecto.

He ido añadiendo comentarios en cada uno de los códigos para que se sepa bien de qué realmente va el código y que se hace en cada uno y para qué sirve cada uno de ellos.

Arquitectura general

El proyecto se ha desarrollado siguiendo una arquitectura modular basada en programación orientada a objetos. Todas las funcionalidades principales del sistema se organizan en clases independientes, estructuradas en archivos `.hpp` (definición) y `.cpp` (implementación).

Se ha diseñado una jerarquía de clases donde los elementos principales de la escena heredan de clases base comunes, permitiendo reutilización de código y una mejor organización. Entre las clases más importantes destacan:

- **Mesh**: Representación de geometría 3D.
- **Model**: Encapsulación de modelos complejos.
- **Camera**: Control de visualización.
- **Light**: Gestión de iluminación.
- **Texture/material**: Aplicación de materiales y texturas.
- **Skybox**: Fondo de la escena.
- **Terrain**: El terreno de la escena, malla de elevación.
- **Node**: Base del grafo de escena.
- **Scene**: Gestión global.

El sistema permite separar claramente la lógica de renderizado, la gestión de recursos y el control de la escena, facilitando la escalabilidad del proyecto.


```

▷ Camera.hpp
▷ Cube.hpp
▷ Light.hpp
▷ Material.hpp
▷ Mesh.hpp
▷ Model.hpp
▷ Node.hpp
▷ Scene.hpp
▷ Shader_Program.hpp
▷ Skybox.hpp
▷ Terrain.hpp
▷ Texture_Cube.hpp
▷ Texture2D.hpp
▷ Transform.hpp
▷ Window.hpp

```

Aquí tenemos los .hpp de todos los elementos que he ido implementando a medida que he ido haciendo la escena.

```

▷ Camera.cpp
▷ Cube.cpp
▷ Light.cpp
▷ main.cpp
▷ Material.cpp
▷ Mesh.cpp
▷ Model.cpp
▷ Node.cpp
▷ Scene.cpp
▷ Shader_Program.cpp
▷ Skybox.cpp
▷ Terrain.cpp
▷ Texture_Cube.cpp
▷ Texture2D.cpp
▷ Transform.cpp
▷ Window.cpp

```

Y aquí están los .cpp de cada uno de los elementos de la escena donde se llegan a completar y a crear todas las funciones por completo, es decir, no solo se definen si no que se llegan a formar y a crear todas las funciones que hemos definido anteriormente en los .hpp.

Control de cámara

La navegación dentro del entorno tridimensional se gestiona mediante una cámara virtual implementada en la clase `Camera`. Este sistema sigue el modelo de cámara en primera persona (First Person Shooter, FPS), permitiendo al usuario desplazarse libremente por la escena y controlar su orientación en tiempo real.

Fundamentos matemáticos

El sistema de cámara se basa en el cálculo dinámico de dos matrices fundamentales en el pipeline gráfico:

- **Matriz de Vista (View Matrix):**

Se encarga de transformar las coordenadas del mundo al espacio de la cámara. Para su cálculo se utiliza la función `LookAt`, definida a partir de tres vectores ortogonales:

- **Posición** de la cámara
- **Dirección frontal (Front)**
- **Vector superior (Up)**

```
// =====
// VIEW MATRIX
// =====
glm::mat4 Camera::get_view_matrix() const
{
    float x, y, z;
    getDirection(x, y, z);

    glm::vec3 front = glm::normalize(glm::vec3(x, y, z));

    glm::vec3 centeredCamPos(0.f, 5.f, -6.f);

    return glm::lookAt(
        position,
        position + front,
        glm::vec3(0, 1, 0)
    );
}
```

- Estos vectores se recalculan continuamente en función de la orientación de la cámara.

```
// =====
// DIRECCIÓN
// =====

/*
    Calcula la dirección hacia donde mira la cámara.
    *
    * x Componente X de la dirección.
    * y Componente Y de la dirección.
    * z Componente Z de la dirección.
    */

void Camera::getDirection(float& x, float& y, float& z) const
{
    x = cos(rotX) * cos(rotY);
    y = sin(rotX);
    z = cos(rotX) * sin(rotY);
}
```

- **Matriz de Proyección (Projection Matrix):**
Se define mediante una proyección en perspectiva, utilizando los siguientes parámetros:
 - Campo de visión (**FOV**)
 - Relación de aspecto de la ventana (**aspect ratio**)
 - Planos de recorte cercano (**near**) y lejano (**far**)
- Esta matriz permite simular la percepción de profundidad, generando una representación visual realista del entorno.

```
// =====
// PROJECTION
// =====

/*Calcula la matriz de proyección en perspectiva en función del campo de visión, ratio de aspecto y planos de recorte.
*/
void Camera::updateProjection()
{
    projection_matrix = glm::perspective(
        glm::radians(fov),
        ratio,
        near_z,
        far_z
    );
}
```

```
// =====
// PROJECTION GETTER
// =====

/*
    Devuelve la matriz de proyección.
    * Referencia constante a la matriz de proyección.
*/
const glm::mat4& Camera::get_projection_matrix() const
{
    return projection_matrix;
}

// =====
// SETTERS
// =====

//Establece la posición de la cámara
void Camera::setPosition(float x, float y, float z)
{
    position = { x, y, z };
}

//Establece la velocidad del movimiento
void Camera::setSpeed(float s) { speed = s; }

//Establece la sensibilidad del ratón
void Camera::setSensitivity(float s) { sensitivity = s; }

//Establece el ratio de aspecto y actualiza la proyección
void Camera::setRatio(float r)
{
    ratio = r;
    updateProjection();
}
```

Sistema de control

El control de la cámara se realiza en tiempo real mediante la interacción con dispositivos de entrada, concretamente teclado y ratón.

- **Movimiento (teclado):**

El desplazamiento se implementa mediante las teclas:

- **W / S:** avance y retroceso
- **A / D:** movimiento lateral
- La posición de la cámara se actualiza en función de sus vectores directores (Front y Right), garantizando que el movimiento sea relativo a la orientación actual. Además, se utiliza un factor de tiempo (DeltaTime) para asegurar una velocidad constante independientemente de la tasa de fotogramas (FPS).

- **Orientación (ratón):**

La rotación de la cámara se controla mediante el movimiento del cursor, que modifica los ángulos:

- **Yaw:** rotación horizontal
- **Pitch:** rotación vertical

```
// =====  
// ROTACIÓN  
// =====  
  
/*  
    Rota la cámara según el movimiento del ratón.  
*  
* dx Movimiento en el eje X.  
* dy Movimiento en el eje Y.  
*/  
void Camera::rotate(float dx, float dy)  
{  
    rotY += dx * sensitivity;  
    rotX -= dy * sensitivity;  
  
    const float TWO_PI = 6.28318f;  
    if (rotY > TWO_PI) rotY -= TWO_PI;  
    if (rotY < 0.0f)   rotY += TWO_PI;  
  
    const float limit = 1.55f;  
    rotX = glm::clamp(rotX, -limit, limit);  
}
```

- A partir de estos ángulos se recalcula el vector dirección (Front). Para evitar problemas como el **gimbal lock**, el ángulo de Pitch se restringe dentro de un rango limitado, impidiendo que la cámara se invierta.

```
// =====  
// VIEW MATRIX  
// =====  
  
/*  
    Obtiene la matriz de vista.  
    * Matriz de vista calculada con glm::lookAt.  
*/  
glm::mat4 Camera::get_view_matrix() const  
{  
    float x, y, z;  
    getDirection(x, y, z);  
  
    glm::vec3 front = glm::normalize(glm::vec3(x, y, z));  
  
    return glm::lookAt(  
        position,  
        position + front,  
        glm::vec3(0, 1, 0)  
    );  
}
```

Representación espacial

El sistema de cámara se apoya en un conjunto de vectores que definen su orientación en el espacio tridimensional:

- **Front:** dirección hacia la que mira la cámara
- **Right:** vector lateral, perpendicular a Front
- **Up:** vector vertical, perpendicular al plano formado por Front y Right

Estos vectores forman una base ortonormal que permite calcular correctamente tanto el movimiento como la orientación.

Integración en la escena

Durante cada fotograma, la cámara actualiza sus matrices de vista en función de la entrada del usuario. Estas matrices se envían al shader como uniformes, permitiendo transformar correctamente todos los objetos de la escena desde el espacio de mundo hasta el espacio de cámara.

Este enfoque proporciona un sistema de navegación fluido, preciso y desacoplado del resto de componentes, facilitando su reutilización y ampliación en futuras mejoras del proyecto.

```
// =====
// ===== CONTROLS =====
// =====

void Scene::moveForward(float dt) { camera.moveForward(dt); }
void Scene::moveBackward(float dt) { camera.moveBackward(dt); }
void Scene::moveLeft(float dt) { camera.moveLeft(dt); }
void Scene::moveRight(float dt) { camera.moveRight(dt); }
void Scene::moveUp(float dt) { camera.moveUp(dt); }
void Scene::moveDown(float dt) { camera.moveDown(dt); }
void Scene::rotateCamera(float dx, float dy) { camera.rotate(dx, dy); }

void Scene::handleMouse(float dx, float dy, float dt)
{
    camera.rotate(dx, dy);

    const float speed = 5.0f * dt;

    float dirX, dirY, dirZ;
    camera.getDirection(dirX, dirY, dirZ);

    glm::vec3 forward = glm::normalize(glm::vec3(dirX, dirY, dirZ));
    glm::vec3 right = glm::normalize(glm::cross(forward, glm::vec3(0, 1, 0)));

    if (fabs(dy) > 1.0f)
    {
        camera.setPosition(
            camera.getX() + forward.x * speed * (dy < 0 ? 1 : -1),
            camera.getY() + forward.y * speed * (dy < 0 ? 1 : -1),
            camera.getZ() + forward.z * speed * (dy < 0 ? 1 : -1)
        );
    }

    if (fabs(dx) > 1.0f)
    {
        camera.setPosition(
            camera.getX() + right.x * speed * (dx > 0 ? 1 : -1),
            camera.getY() + right.y * speed * (dx > 0 ? 1 : -1),
            camera.getZ() + right.z * speed * (dx > 0 ? 1 : -1)
        );
    }
}
```

Mallas de elevación

Para la representación del entorno tridimensional, se ha implementado un sistema de generación procedural de terreno basado en mapas de elevación (*heightmaps*), encapsulado en la clase Terrain. Este enfoque permite construir superficies complejas de forma eficiente a partir de datos bidimensionales.

Principio de funcionamiento

El sistema utiliza una imagen en escala de grises como entrada, donde cada píxel representa un punto del terreno:

- Las coordenadas del píxel determinan su posición en el plano horizontal (**X**, **Z**)
- La intensidad del color (valor entre 0.0 y 1.0) define su altura (**Y**)

De este modo, se transforma una imagen bidimensional en una malla tridimensional con relieve, permitiendo generar estructuras como montañas, colinas o superficies irregulares.

Generación de la geometría

El proceso de construcción del terreno consta de varias etapas:

- **Lectura del mapa de elevación:**
Se recorre la imagen píxel a píxel para extraer los valores de altura normalizados.
- **Cálculo de vértices:**
Para cada punto se genera un vértice tridimensional con coordenadas (X, Y, Z), donde Y se obtiene a partir del valor de intensidad del píxel escalado por un factor de altura.
- **Coordenadas de textura (UV):**
Se asignan coordenadas UV a cada vértice, permitiendo aplicar texturas repetidas sobre la superficie del terreno.
- **Cálculo de normales:**
Las normales se calculan de forma matemática a partir de los vértices adyacentes, lo que resulta esencial para una iluminación realista basada en shaders.

```
void Terrain::Draw() const
{
    if (!vao_id) return;
    glBindVertexArray(vao_id);
    glDrawElements(GL_TRIANGLES, index_count, GL_UNSIGNED_INT, 0);
    glBindVertexArray(0);
}

// Genera alturas mas altas y con un poco de ruido
void Terrain::GenerateHeightmap()
{
    m_heights.resize(m_width * m_height);

    // Generador de ruido aleatorio pequeño
    std::mt19937 rng(42); // semilla fija
    std::uniform_real_distribution<float> noise(-0.2f, 0.2f);

    for (int z = 0; z < m_height; ++z)
    {
        for (int x = 0; x < m_width; ++x)
        {
            // Elevación mas marcada
            float height = (std::sin(x * 0.1f) * std::cos(z * 0.2f) * 7.0f + noise(rng)) * m_scale;
            m_heights[z * m_width + x] = height;
        }
    }
}
```

```

float Terrain::GetHeight(int x, int z) const
{
    return m_heights[z * m_width + x];
}

// Calcula normales
glm::vec3 Terrain::CalculateNormal(int x, int z) const
{
    float hL = (x > 0) ? GetHeight(x - 1, z) : GetHeight(x, z);
    float hR = (x < m_width - 1) ? GetHeight(x + 1, z) : GetHeight(x, z);
    float hD = (z > 0) ? GetHeight(x, z - 1) : GetHeight(x, z);
    float hU = (z < m_height - 1) ? GetHeight(x, z + 1) : GetHeight(x, z);

    glm::vec3 normal(hL - hR, 2.0f, hD - hU);
    return glm::normalize(normal);
}

// Construye vertices e indices
void Terrain::BuildMesh()
{
    m_vertices.clear();
    m_indices.clear();

    for (int z = 0; z < m_height; ++z)
    {
        for (int x = 0; x < m_width; ++x)
        {
            float y = GetHeight(x, z);
            glm::vec3 pos(x * m_scale, y, z * m_scale);
            glm::vec3 normal = CalculateNormal(x, z);
            glm::vec2 uv(float(x) / m_width, float(z) / m_height);
            m_vertices.emplace_back(pos, normal, uv);
        }
    }

    for (int z = 0; z < m_height - 1; ++z)
    {
        for (int x = 0; x < m_width - 1; ++x)
        {
            int topLeft = z * m_width + x;
            int topRight = topLeft + 1;
            int bottomLeft = (z + 1) * m_width + x;
            int bottomRight = bottomLeft + 1;

            m_indices.push_back(topLeft);
            m_indices.push_back(bottomLeft);
            m_indices.push_back(topRight);

            m_indices.push_back(topRight);
            m_indices.push_back(bottomLeft);
            m_indices.push_back(bottomRight);
        }
    }
}

```

Optimización mediante indexación

Para mejorar el rendimiento, la geometría del terreno se gestiona utilizando un **Element Buffer Object (EBO)**:

- Se evita la duplicación de vértices
- Se reutilizan índices para construir los triángulos
- Cada celda de la rejilla se divide en dos triángulos

Este enfoque reduce significativamente el uso de memoria y el número de datos transferidos entre CPU y GPU, optimizando el rendimiento en tiempo de ejecución.

Mejora del realismo

Con el objetivo de evitar superficies excesivamente uniformes, se introduce una variación adicional mediante ruido procedural de baja intensidad. Este ruido se aplica sobre los valores de altura, generando irregularidades sutiles que aportan mayor naturalidad al terreno.

Asimismo, se permite ajustar un factor de escala vertical para controlar la altura máxima del relieve, adaptando el terreno a las necesidades visuales de la escena.

Integración en el sistema de renderizado

Una vez generada, la malla del terreno se transfiere a la GPU mediante buffers (VBO/EBO) y se renderiza en cada fotograma utilizando shaders. La textura aplicada se repite mediante técnicas de *tiling*, lo que permite cubrir grandes extensiones sin pérdida de calidad visual.

El terreno actúa como base de la escena, proporcionando un contexto espacial sobre el que se sitúan el resto de objetos, como modelos, luces y elementos animados.

```
// Sube la malla a GPU
void Terrain::UploadToGPU()
{
    std::vector<float> vertex_data;
    for (const auto& v : m_vertices)
    {
        vertex_data.push_back(v.position.x);
        vertex_data.push_back(v.position.y);
        vertex_data.push_back(v.position.z);

        vertex_data.push_back(v.normal.x);
        vertex_data.push_back(v.normal.y);
        vertex_data.push_back(v.normal.z);

        vertex_data.push_back(v.texCoords.x);
        vertex_data.push_back(v.texCoords.y);
    }

    glGenVertexArrays(1, &vao_id);
    glBindVertexArray(vao_id);

    glGenBuffers(1, &vbo_id);
    glBindBuffer(GL_ARRAY_BUFFER, vbo_id);
    glBufferData(GL_ARRAY_BUFFER, vertex_data.size() * sizeof(float), vertex_data.data(), GL_STATIC_DRAW);

    glGenBuffers(1, &ebo_id);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo_id);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, m_indices.size() * sizeof(unsigned int), m_indices.data(), GL_STATIC_DRAW);

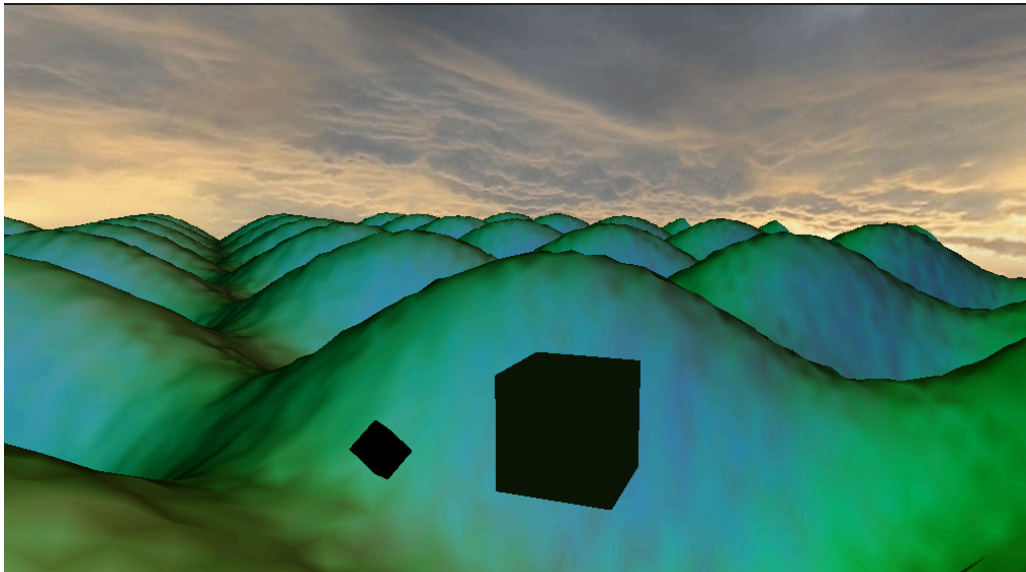
    // Posición
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)0);
    glEnableVertexAttribArray(0);

    // Normal
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)(3 * sizeof(float)));
    glEnableVertexAttribArray(1);

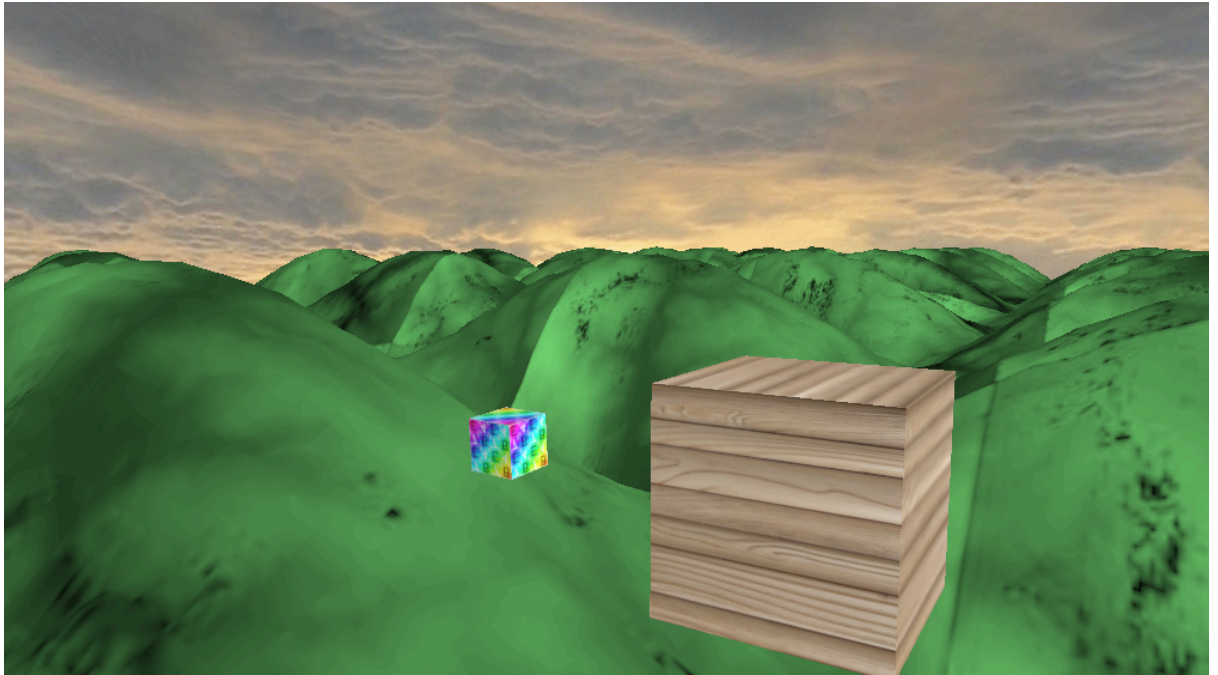
    // UV
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)(6 * sizeof(float)));
    glEnableVertexAttribArray(2);

    glBindVertexArray(0);

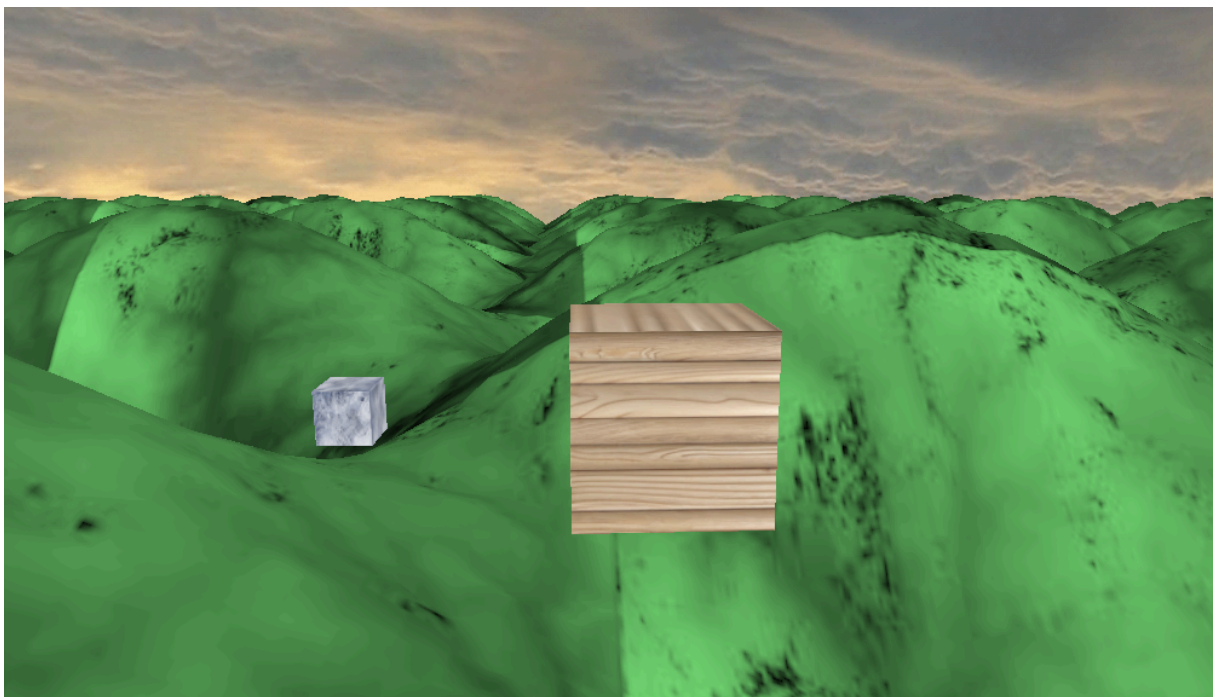
    index_count = static_cast<GLsizei>(m_indices.size());
}
```



Así es como estaba al principio de implementar todas las texturas.



Aquí es donde conseguí que se añadieran las texturas.



Así es como queda en la escena final.

Grafo de escena

Para la organización de los elementos tridimensionales, se ha implementado un **grafo de escena** (*scene graph*), una estructura jerárquica ampliamente utilizada en motores gráficos para gestionar objetos y sus transformaciones de forma eficiente.

Estructura del sistema

El grafo de escena se compone de nodos, representados mediante la clase **Node**. Cada nodo encapsula la información necesaria para su representación y relación con otros elementos:

- **Transformación local:** matriz que define la posición, rotación y escala del nodo respecto a su padre
- **Lista de hijos:** colección de nodos dependientes jerárquicamente
- **Objeto asociado:** referencia opcional a una malla (**Mesh**) o modelo (**Model**) a renderizar

Esta estructura permite construir una escena como un árbol, donde cada nodo puede tener múltiples hijos pero un único padre.

Transformaciones jerárquicas

Una de las principales ventajas del grafo de escena es la propagación de transformaciones. La transformación global de un nodo se obtiene combinando su transformación local con la de sus ancestros:

- Si un nodo padre se traslada, rota o escala, todos sus nodos hijos heredan automáticamente dicha transformación
- Esto permite modelar relaciones espaciales complejas de forma sencilla

Por ejemplo, un objeto situado sobre el terreno se moverá automáticamente si el nodo del terreno cambia su transformación, sin necesidad de recalcular manualmente su posición.

Recorrido y renderizado

El proceso de renderizado se realiza mediante un recorrido del grafo, generalmente en profundidad (*depth-first traversal*). Durante este proceso:

1. Se acumulan las transformaciones desde la raíz hasta el nodo actual
2. Se calcula la matriz de modelo global
3. Se envía dicha matriz al shader
4. Se renderiza el objeto asociado, si existe

5. Se continúa recursivamente con los nodos hijos

Este enfoque permite aplicar correctamente todas las transformaciones jerárquicas en tiempo real.

Ventajas del enfoque

El uso de un grafo de escena aporta múltiples beneficios:

- **Organización estructurada:** facilita la gestión de escenas complejas
- **Reutilización de código:** permite aplicar lógica común a múltiples nodos
- **Escalabilidad:** simplifica la incorporación de nuevos elementos
- **Eficiencia:** reduce la necesidad de recalcular transformaciones manualmente

Integración en la escena

En el sistema desarrollado, el grafo de escena actúa como base organizativa sobre la que se construyen los distintos elementos del entorno. Por ejemplo:

- Un nodo raíz representa la escena global
- El terreno se define como un nodo principal
- Otros objetos, como los cubos animados, pueden añadirse como nodos hijos

Este diseño permite mantener una estructura clara y extensible, alineada con los principios utilizados en motores gráficos modernos.

Scene

La clase **Scene** constituye el núcleo del sistema de renderizado, siendo la encargada de gestionar todos los elementos visibles, inicializar OpenGL, cargar recursos y ejecutar el bucle de dibujo en cada fotograma.

Inicialización de la escena

En el constructor de la clase Scene se realiza toda la configuración inicial del entorno gráfico:

- **Carga del skybox** mediante un cubemap:

```
skybox = std::make_shared<Skybox>("../shared/assets/sky-cube-map-");
```

- **Carga de texturas**, como la textura de madera:

```
texture_wood = make_shared<Texture2D>("../shared/assets/wood.png");
```

- **Activación de estados de OpenGL:**
 - GL_CULL_FACE → descarte de caras traseras
 - GL_DEPTH_TEST → test de profundidad
 - GL_BLEND → transparencia
- **Configuración del color de fondo:**

```
glClearColor(0.2f, 0.4f, 0.9f, 1.0f);
```

- **Compilación y enlace de shaders**
- **Obtención de uniforms** (matrices, texturas, etc.)

Shaders e iluminación

La escena utiliza un pipeline programable basado en shaders GLSL.

Vertex Shader

Se encarga de:

- Transformar vértices al espacio de clip
- Calcular normales transformadas
- Pasar información al fragment shader:
 - posición (frag_pos)
 - normal (normal)
 - coordenadas UV

Además, se genera un color base en función de la normal para depuración.

Fragment Shader

Implementa un modelo de iluminación basado en:

- **Componente ambiental**
- **Difusa (Lambert)**
- **Especular (Phong)**

```
vec3 result = ambient + diffuse + specular;
```

También incluye:

- **Corrección gamma** (`pow(result, vec3(1.3))`)
- **Soporte para texturas** mediante `diffuse_map`
- Uso de `uv_scale` para repetir texturas

Estructura de la escena

La escena está compuesta por varios elementos principales:

1. Terreno (Terrain)

- Posicionado mediante:

```
glm::translate(l, TERRAIN_OFFSET);
```

- Utiliza una textura repetida (tileado):

```
glUniform1f(..., 8.0f);
```

- Se renderiza mediante:

```
terrain.Draw();
```

El terreno actúa como base del entorno, representando un paisaje tridimensional.

2. Cubo principal

- Posicionado en:

```
MAIN_CUBE_POS
```

- Escalado:

```
MAIN_CUBE_SCALE
```

- Rotación animada:

```
angle * 0.5f
```

- Texturizado con una textura tipo “tierra”:

```
Texture2D::get("earth");
```

Se trata del objeto central de la escena, con una rotación suave para aportar dinamismo.

3. Cubo secundario (animado)

Este cubo introduce una animación más compleja:

- Movimiento orbital alrededor del cubo principal
- Rotación sobre sí mismo
- Escalado reducido

Transformaciones aplicadas:

translate → rotate → translate → rotate → scale

Además:

- Tiene **transparencia (alpha = 0.5)**
- Su color cambia dinámicamente con funciones trigonométricas:

$\cos(\text{angle})$, $\sin(\text{angle})$

Esto genera un efecto visual dinámico y atractivo.

Animación

La animación global de la escena se controla mediante:

$\text{angle} += 0.01f$;

Este valor se utiliza para:

- Rotación del cubo principal
- Movimiento orbital del cubo pequeño
- Cambio de color dinámico

Sistema de renderizado

El método `render()` sigue este flujo:

1. Limpieza de buffers:

```
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
```

2. Renderizado del skybox
3. Activación del shader principal
4. Renderizado de:
 - Terreno
 - Cubo principal
 - Cubo secundario
5. Gestión de texturas y transparencias

Transformaciones

Se utiliza una función auxiliar:

`make_model`

Que encapsula:

- Traslación
- Rotación
- Escalado

Esto permite generar matrices de modelo de forma limpia y reutilizable.

Gestión de cámara

La escena utiliza la cámara para obtener la matriz de vista:

```
glm::mat4 view = camera.get_view_matrix();
```

Además, se integran controles de usuario:

- Movimiento (WASD, etc.)
- Rotación con ratón
- Movimiento relativo a la dirección de la cámara

Proyección

La proyección se define en `resize()`:

```
glm::perspective(  
    glm::radians(45.f),  
    aspect_ratio,  
    1.f,  
    5000.f  
);
```

Esto permite una visualización en perspectiva realista con gran distancia de renderizado.

Conclusión de la escena

La clase Scene implementa un sistema completo de renderizado que integra:

- Iluminación avanzada
- Texturas
- Transparencia
- Animación
- Skybox
- Control de cámara

La organización modular y el uso de shaders permiten una representación eficiente y visualmente rica del entorno 3D.

```
// =====
// CONSTRUCTOR & DESTRUCTOR
// =====

Scene::Scene(unsigned width, unsigned height)
: angle(0.0f), moon_angle(0.0f)
{
    skybox = std::make_shared<Skybox>("../shared/assets/sky-cube-map-");
    load_textures();

    glEnable(GL_CULL_FACE);
    glEnable(GL_DEPTH_TEST);
    glDepthFunc(GL_LEQUAL);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
    glClearColor(0.2f, 0.4f, 0.9f, 1.0f);

    program_id = compile_shaders();
    glUseProgram(program_id);

    model_view_matrix_id = glGetUniformLocation(program_id, "model_view_matrix");
    projection_matrix_id = glGetUniformLocation(program_id, "projection_matrix");

    glUniform1i(glGetUniformLocation(program_id, "use_texture"), 1);
    glUniform1i(glGetUniformLocation(program_id, "diffuse_map"), 0);
    glUniform1f(glGetUniformLocation(program_id, "uv_scale"), 1.0f);
    glUniform1i(glGetUniformLocation(program_id, "is_terrain"), 0); // Falso por defecto

    resize(width, height);

    terrain_node = std::make_shared<Model>();
    earth_node = std::make_shared<Model>();
    moon_node = std::make_shared<Model>();

    terrain_node->transform.position = TERRAIN_OFFSET;
    earth_node->transform.position = MAIN_CUBE_POS;
    earth_node->transform.scale = glm::vec3(MAIN_CUBE_SCALE);
    moon_node->transform.position = SMALL_CUBE_OFFSET;
    moon_node->transform.scale = glm::vec3(SMALL_CUBE_SCALE);

    moon_node->set_parent(earth_node.get());
    earth_node->set_parent(&root);
    terrain_node->set_parent(&root);
}
```

Este es el constructor de la clase donde añadimos skybox y las texturas.

```
// =====
// RENDERIZADO DE LA ESCENA (RENDER)
// =====

void Scene::render()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    skybox->render(camera);

    glUseProgram(program_id);
    glm::mat4 view = camera.get_view_matrix();

    glUniform3f(glGetUniformLocation(program_id, "light_pos"), 10.f, 10.f, 10.f);
    glUniform3f(glGetUniformLocation(program_id, "view_pos"), camera.getX(), camera.getY(), camera.getZ());

    root.traverse(0.016f);

    int useTexLoc = glGetUniformLocation(program_id, "use_texture");
    int uvScaleLoc = glGetUniformLocation(program_id, "uv_scale");
    int isTerrainLoc = glGetUniformLocation(program_id, "is_terrain"); // Localización del flag de terreno
}
```

En esta parte es donde renderizamos y donde creamos el terreno con la uvs para el color y para la posición en la que va a estar situado.

```
// -----
// RENDER: CUBO GRANDE (TIERRA)
// -----
glUniform1i(isTerrainLoc, 0); // DESACTIVAMOS el tinte verde para los cubos

auto earthTex = Texture2D::get("earth");
if (earthTex)
{
    cube.set_texture(earthTex->get_id());
    cube.enable_texture(true);

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, earthTex->get_id());

    glUniform1i(useTexLoc, 1);
    glUniform1f(uvScaleLoc, 1.0f);
}

glm::mat4 main_cube_model = make_model(
    glm::mat4(1.0f),
    MAIN_CUBE_POS,
    angle * 0.5f,
    { 0, 1, 0 },
    glm::vec3(MAIN_CUBE_SCALE)
);

glUniformMatrix4fv(model_view_matrix_id, 1, GL_FALSE, glm::value_ptr(view * main_cube_model));
cube.set_color(glm::vec4(1.0f));
cube.render();

glBindTexture(GL_TEXTURE_2D, 0);
```

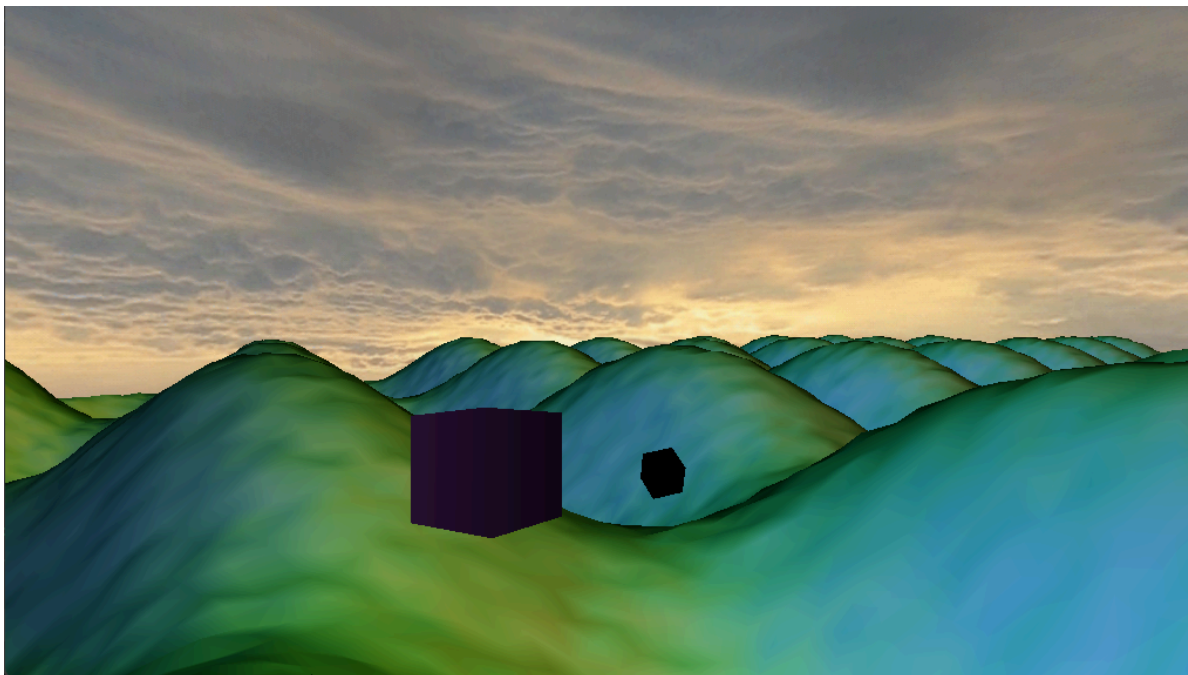
Aquí tenemos definido al cubo grande donde le he puesto algún dato más para que sea mucho más realista y para que cuando gire sobre sí mismo no vaya tan tan rápido, junto con la textura seleccionada.

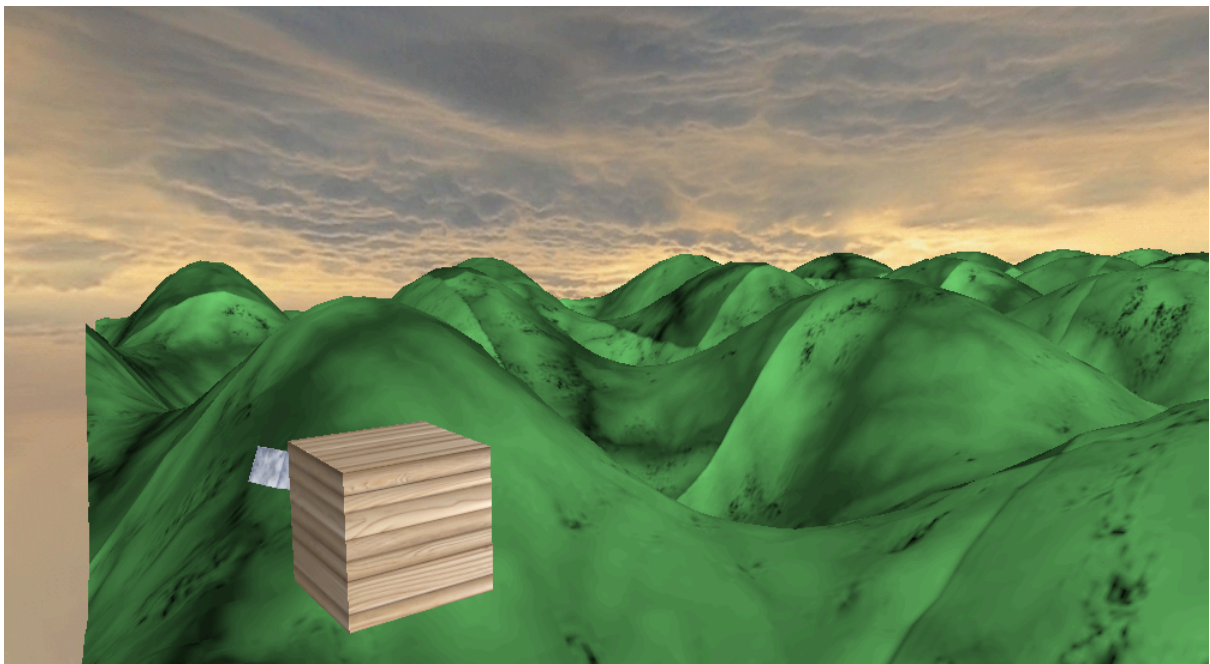
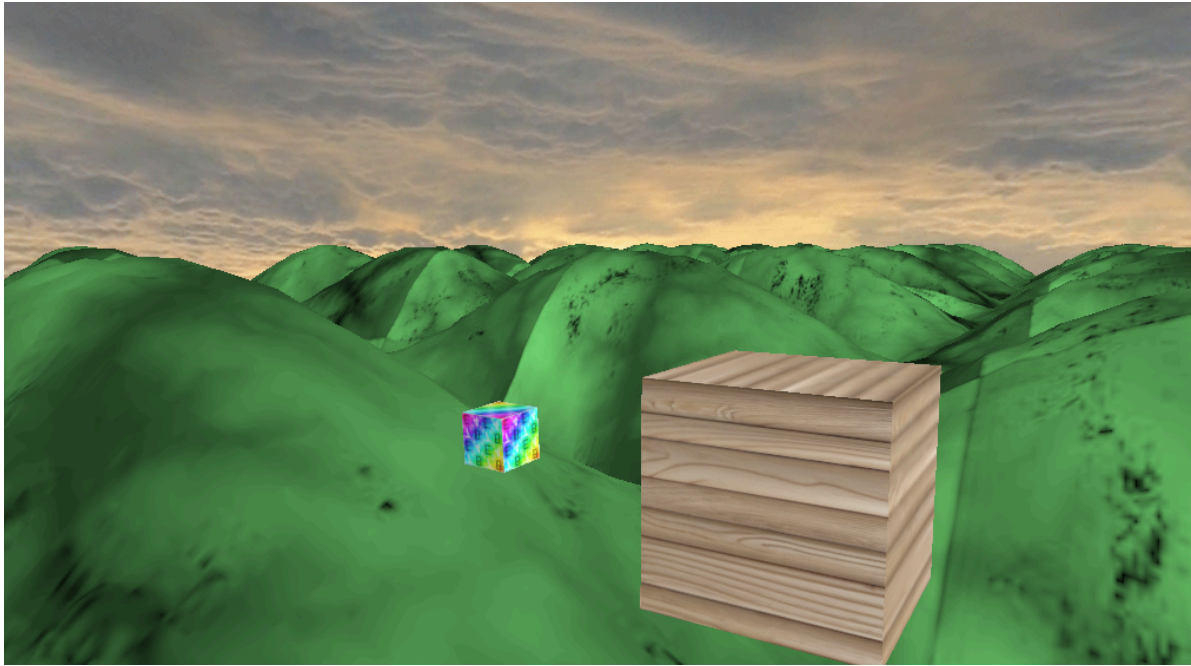
```
// -----  
// RENDER: CUBO PEQUEÑO (LUNA)  
// -----  
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);  
glDepthMask(GL_TRUE);  
  
glm::mat4 small_cube_model = glm::translate(glm::mat4(1.0f), SMALL_CUBE_BASE_POS);  
small_cube_model = glm::rotate(small_cube_model, angle * 1.f, { 0, 1, 0 });  
small_cube_model = glm::translate(small_cube_model, SMALL_CUBE_OFFSET);  
small_cube_model = glm::rotate(small_cube_model, angle * 2.f, { 1, 1, 0 });  
small_cube_model = glm::scale(small_cube_model, glm::vec3(SMALL_CUBE_SCALE));  
  
glUniformMatrix4fv(model_view_matrix_id, 1, GL_FALSE, glm::value_ptr(view * small_cube_model));  
  
cube.set_color(glm::vec4(  
    0.5f + 0.5f * cos(angle * 0.4f),  
    0.5f + 0.5f * sin(angle * 0.25f),  
    0.5f + 0.5f * cos(angle * 0.3f),  
    1.0f  
));  
  
auto moonTex = Texture2D::get("luna");  
if (moonTex)  
{  
    cube.set_texture(moonTex->get_id());  
    cube.enable_texture(true);  
  
    glActiveTexture(GL_TEXTURE0);  
    glBindTexture(GL_TEXTURE_2D, moonTex->get_id());  
  
    glUniform1i(useTexLoc, 1);  
    glUniform1f(uvScaleLoc, 1.0f);  
}  
  
cube.render();  
  
glBindTexture(GL_TEXTURE_2D, 0);  
glDisable(GL_BLEND);  
root.traverse(0.0f);
```

Aquí estamos definiendo lo mismo que el cubo grande solo que en este caso para el cubo pequeño que gira y rota mucho más rápido que el grande y sobre sí mismo pero girado hacia un lado, tambien le he implementado la transparencia junto con el mismo tono de metálico que el cubo grande, es decir, ambos fragment shader y vertex shader son lo mismo para ambos objetos.

```
// =====  
// ENTRADAS, CONTROLES Y RESPUESTA DEL SISTEMA  
// =====  
  
void Scene::moveForward(float dt) { camera.moveForward(dt); }  
void Scene::moveBackward(float dt) { camera.moveBackward(dt); }  
void Scene::moveLeft(float dt) { camera.moveLeft(dt); }  
void Scene::moveRight(float dt) { camera.moveRight(dt); }  
void Scene::moveUp(float dt) { camera.moveUp(dt); }  
void Scene::moveDown(float dt) { camera.moveDown(dt); }  
void Scene::rotateCamera(float dx, float dy) { camera.rotate(dx, dy); }  
  
void Scene::handleMouse(float dx, float dy, float dt)  
{  
    camera.rotate(dx, dy);  
  
    float x, y, z;  
    camera.getDirection(x, y, z);  
  
    glm::vec3 forward = glm::normalize(glm::vec3(x, y, z));  
    glm::vec3 right = glm::normalize(glm::cross(forward, glm::vec3(0, 1, 0)));  
    float speed = 5.0f * dt;  
  
    if (fabs(dy) > 1.0f)  
        camera.setPosition(camera.getX() + forward.x * speed,  
                            camera.getY() + forward.y * speed,  
                            camera.getZ() + forward.z * speed);  
  
    if (fabs(dx) > 1.0f)  
        camera.setPosition(camera.getX() + right.x * speed,  
                            camera.getY() + right.y * speed,  
                            camera.getZ() + right.z * speed);  
}
```

He añadido también los controles de la cámara, para que gire, para que rote, para que se mueva en cualquier momento el usuario.





Y así es como quedaría la escena total con todo implementado, con texturas, material, controles para moverse por toda la escena de manera libre, el skybox...

Texturas/transparencias

Se han aplicado texturas para mejorar el realismo visual de la escena.

Texturas

- Aplicación mediante coordenadas UV
- Uso de imágenes externas cargadas en GPU
- **Tipos utilizados:**
 - Difusas
 - (Opcionalmente) especulares

Transparencia

Se ha implementado el uso de transparencia mediante:

- Activación de **Blending**:

```
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

- Uso de texturas con canal alpha

Esto permite representar elementos como vegetación o superficies translúcidas.

Lo que he hecho para la parte de las texturas es crear una clase Texture 2D.hpp y .cpp para poder llevar a cabo las texturas en cada objeto y en cada parte de la entrega ya sea en los cubos o en el terreno para que se vea mucho más realista.

```
// ===== TEXTURE BANK API =====
//Carga una textura y la guarda en el banco global.

Texture2D* Texture2D::load(const std::string& name, const std::string& path)
{
    if (texture_bank.count(name))
        return texture_bank[name];

    Texture2D* tex = new Texture2D(path);

    if (tex->is_loaded())
    {
        texture_bank[name] = tex;
        return tex;
    }

    delete tex;
    return nullptr;
}

Texture2D* Texture2D::get(const std::string& name)//Obtiene una textura del banco global.
{
    auto it = texture_bank.find(name);

    if (it != texture_bank.end())
        return it->second;

    std::cerr << "[WARN] Textura no encontrada: " << name << std::endl;
    return nullptr;
}
```

```
// ===== BIND =====
//Activa la textura en un slot de textura.

void Texture2D::bind(unsigned int slot) const
{
    if (!loaded || texture_id == 0) return;

    glActiveTexture(GL_TEXTURE0 + slot);
    glBindTexture(GL_TEXTURE_2D, texture_id);
}
```

Aquí es donde he añadido un bind para que se activen las texturas y el texture_id junto con el cache de texturas para poderlas cargar en función del nombre y de la ruta en la que está, es decir, la carpeta donde están situadas las texturas fuera de lo que es el proyecto. En este caso están en la carpeta de shared/textures.

Y estas texturas van a estar en el terreno y ambos cubos para que se diferencien del resto de la escena, ya que los cubos ambos son de un tono metálico, solo que el pequeño tiene aplicada la transparencia.

```
void Texture2D::load_default_textures() // Carga texturas por defecto del motor
{
    std::cout << "Cargando texturas...\n";

    // =====
    // TEXTURAS DE LA ESCENA
    // =====

    // Terreno (suelo)
    load("terrain", "../../shared/assets/terrain.png");

    // Cubo grande (TIERRA)
    load("earth", "../../shared/assets/Imagen.png");

    // Cubo pequeño (LUNA / ORBITAL)
    load("luna", "../../shared/assets/uv-checker.png");

    // Extra (si lo usas en UI u objetos secundarios)
    load("circulo", "../../shared/assets/circulo-oscuro.png");

    std::cout << "Texturas cargadas.\n";
}
```

Y aquí es donde definimos todas las texturas que queremos tener cargadas en la escena.

```
// =====
// GESTIÓN DE TEXTURAS
// =====

void Scene::load_textures()
{
    Texture2D::load("terrain", "../../shared/assets/Imagen.png");
    Texture2D::load("earth", "../../shared/assets/wood.png");
    Texture2D::load("luna", "../../shared/assets/uv-checker.png");
}
```

Aquí en Scene es donde inicializamos las texturas que vamos a poner en la escena.

```
// -----
// RENDER: TERRENO
// -----
glUniform1i(isTerrainLoc, 1); // ACTIVAMOS el tinte verde en el shader

auto terrainTex = Texture2D::get("terrain");
if (terrainTex)
{
    glActiveTexture(GL_TEXTURE0);
    terrainTex->bind(0);
    glUniform1i(useTexLoc, 1);
    glUniform1f(uvScaleLoc, 8.0f);
}
else
{
    glUniform1i(useTexLoc, 0);
}

glm::mat4 terrain_model = glm::translate(glm::mat4(1.0f), TERRAIN_OFFSET);
glUniformMatrix4fv(model_view_matrix_id, 1, GL_FALSE, glm::value_ptr(view * terrain_model));
terrain.Draw();

glBindTexture(GL_TEXTURE_2D, 0);
```

Luego hacemos un render tanto de la escena como de cada parte por separado, en este caso, es del terreno y de los dos cubos por separado para que se diferencie bien cada parte.

```
// -----
// RENDER: CUBO GRANDE (TIERRA)
// -----
glUniform1i(isTerrainLoc, 0); // DESACTIVAMOS el tinte verde para los cubos

auto earthTex = Texture2D::get("earth");
if (earthTex)
{
    cube.set_texture(earthTex->get_id());
    cube.enable_texture(true);

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, earthTex->get_id());

    glUniform1i(useTexLoc, 1);
    glUniform1f(uvScaleLoc, 1.0f);
}

glm::mat4 main_cube_model = make_model(
    glm::mat4(1.0f),
    MAIN_CUBE_POS,
    angle * 0.5f,
    { 0, 1, 0 },
    glm::vec3(MAIN_CUBE_SCALE)
);

glUniformMatrix4fv(model_view_matrix_id, 1, GL_FALSE, glm::value_ptr(view * main_cube_model));
cube.set_color(glm::vec4(1.0f));
cube.render();

glBindTexture(GL_TEXTURE_2D, 0);
```

Aquí renderizamos la parte del cubo grande

```
// -----
// RENDER: CUBO PEQUEÑO (LUNA)
// -----
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_TRUE);

glm::mat4 small_cube_model = glm::translate(glm::mat4(1.0f), SMALL_CUBE_BASE_POS);
small_cube_model = glm::rotate(small_cube_model, angle * 1.f, { 0, 1, 0 });
small_cube_model = glm::translate(small_cube_model, SMALL_CUBE_OFFSET);
small_cube_model = glm::rotate(small_cube_model, angle * 2.f, { 1, 1, 0 });
small_cube_model = glm::scale(small_cube_model, glm::vec3(SMALL_CUBE_SCALE));

glUniformMatrix4fv(model_view_matrix_id, 1, GL_FALSE, glm::value_ptr(view * small_cube_model));

cube.set_color(glm::vec4(
    0.5f + 0.5f * cos(angle * 0.4f),
    0.5f + 0.5f * sin(angle * 0.25f),
    0.5f + 0.5f * cos(angle * 0.3f),
    1.0f
));

auto moonTex = Texture2D::get("luna");
if (moonTex)
{
    cube.set_texture(moonTex->get_id());
    cube.enable_texture(true);

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, moonTex->get_id());

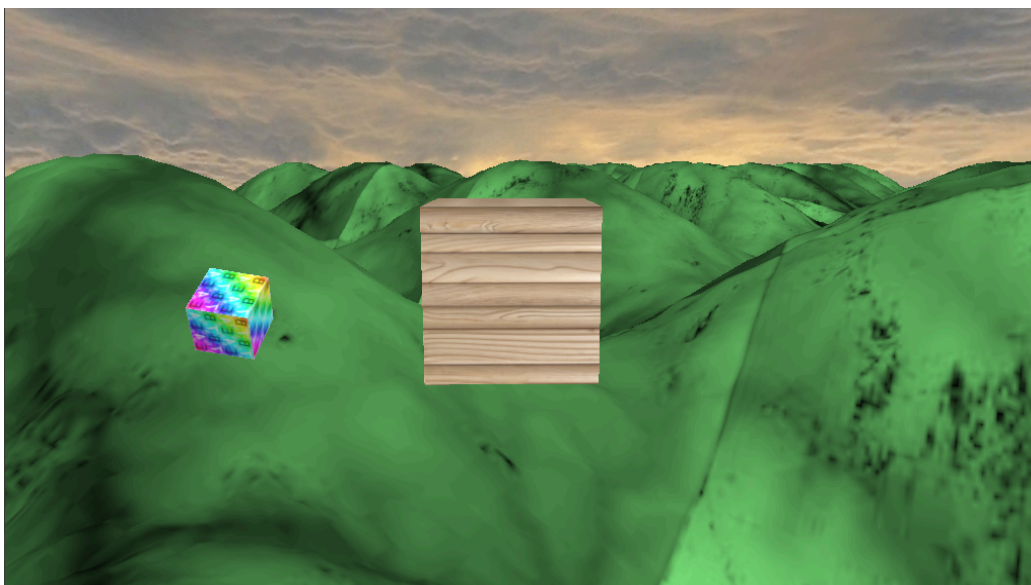
    glUniform1i(useTexLoc, 1);
    glUniform1f(uvScaleLoc, 1.0f);
}

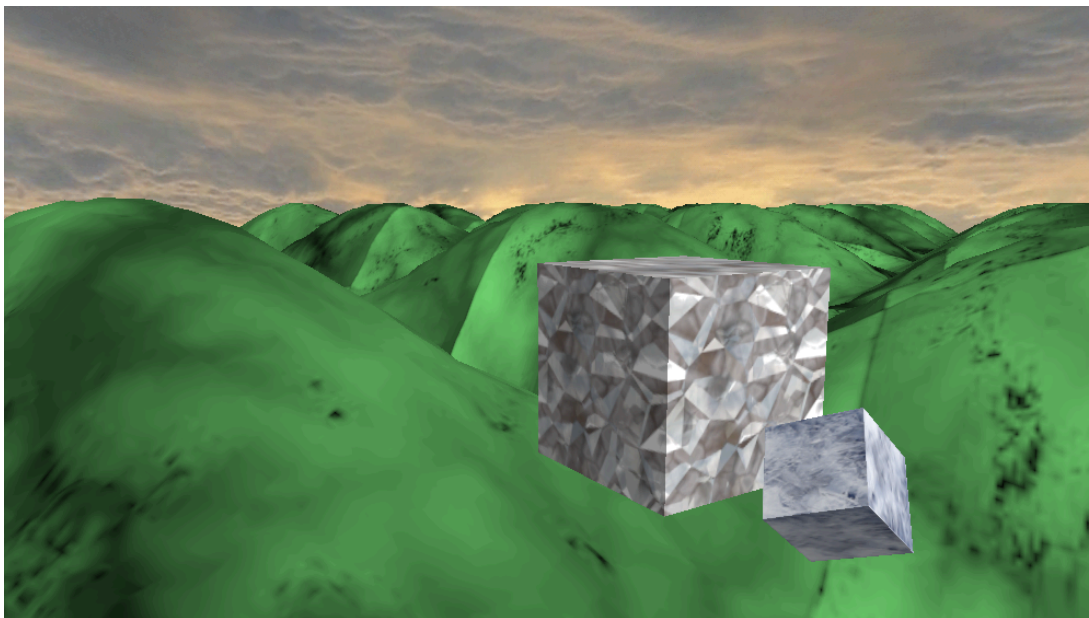
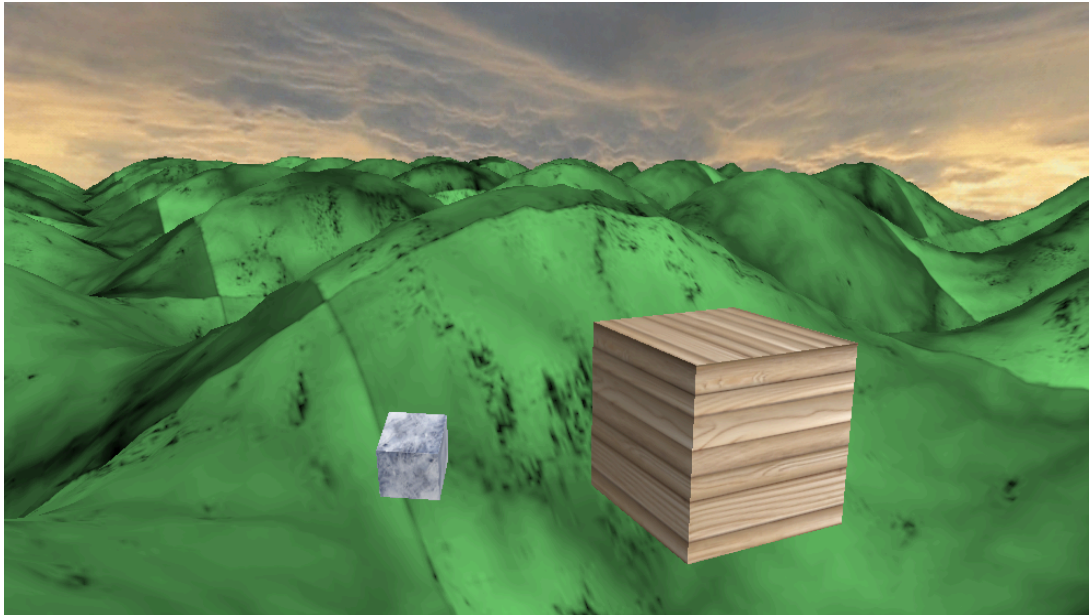
cube.render();

glBindTexture(GL_TEXTURE_2D, 0);
glDisable(GL_BLEND);

root.traverse(0.0f);
```

Y aquí la parte del cubo pequeño (Luna) que es el que gira alrededor del cubo grande como si fuera la luna o un pequeño satélite, junto a la última línea de código que es el grafo de escena.





En caso de que se desee cambiar cualquier tipo de textura en cualquier modelo se puede hacer sin ningún problema.

Para la incorporación de texturas en la escena, se ha aplicado en primer lugar una textura base de color gris al terreno, con el objetivo de simular un suelo con ligeras variaciones que evocan dunas suaves. Sobre esta base, la malla de elevación ha sido complementada con un material de tonalidad verde, aportando un mayor realismo visual y coherencia con el entorno.

En cuanto a los elementos individuales, se han asignado dos texturas completamente diferentes a cada uno de los cubos, con el fin de diferenciarlos visualmente dentro de la escena. Además, al cubo de menor tamaño se le ha añadido un efecto de transparencia, mejorando así la percepción de profundidad y variedad visual.

El proceso de implementación de las texturas ha resultado especialmente complejo, debido a la dificultad inicial para comprender tanto su aplicación como su correcta visualización en la escena y en pantalla. No obstante, tras un proceso de aprendizaje continuo basado en pruebas, errores y ajustes, se ha logrado obtener el resultado final deseado.

Skybox/postproceso

El skybox es la parte del fondo de la escena, es decir, el cielo, las nubes, el sol, todo lo que es externo al núcleo principal.

```
class Skybox
{
public:
    // Constructor: recibe la ruta base del cubemap
    Skybox(const std::string& path);

    // Render del skybox
    void render(const Camera& camera);

    // Destructor
    ~Skybox();
    Skybox* skybox;

private:
    // ===== CARGA DEL CUBEMAP =====
    GLuint loadCubemap(const std::string& base_path);

    // ===== SHADERS =====
    static const char* vertex_shader_code;
    static const char* fragment_shader_code;

    // ===== GEOMETRÍA DEL CUBO =====
    static const GLfloat coordinates[];

    // ===== IDs OpenGL =====
    GLuint vao_id = 0;
    GLuint vbo_id = 0;
    GLuint shader_id = 0;
    GLuint cubemap_texture = 0;
};
```

Lo que he realizado en el .hpp es hacer el constructor, renderizar el skybox que he definido y por último, crearme el cubemap ya que en el escena está compuesto por varias caras como si fuera un cubo con una serie de vértices y de coordenadas.

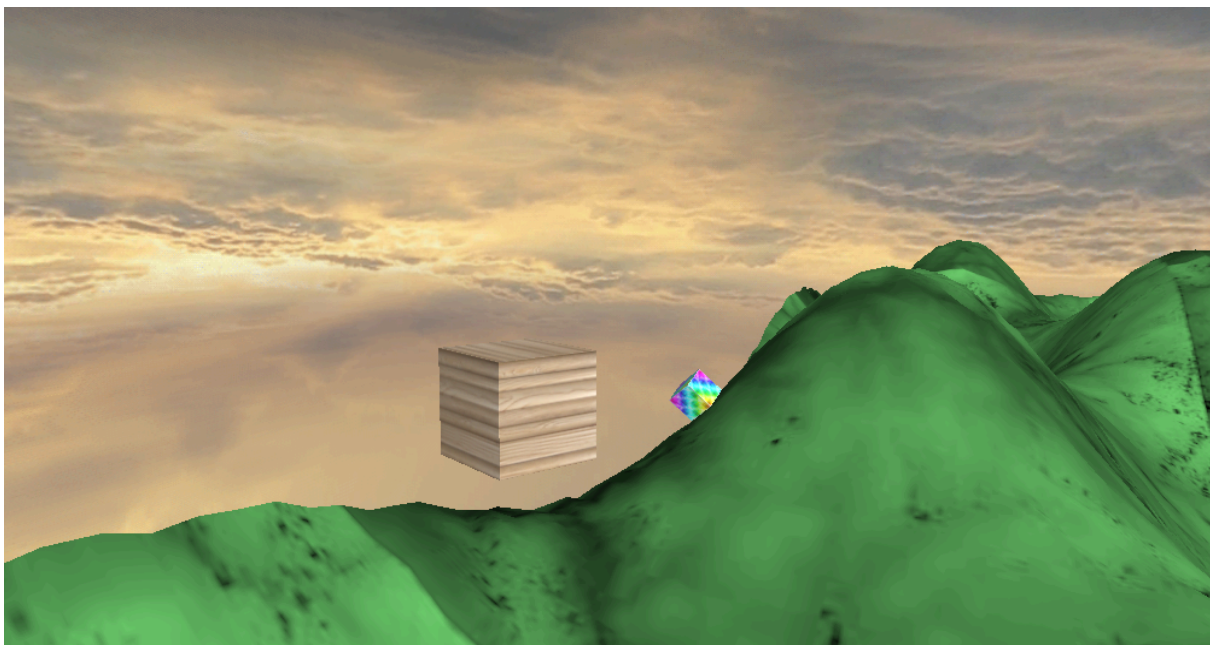
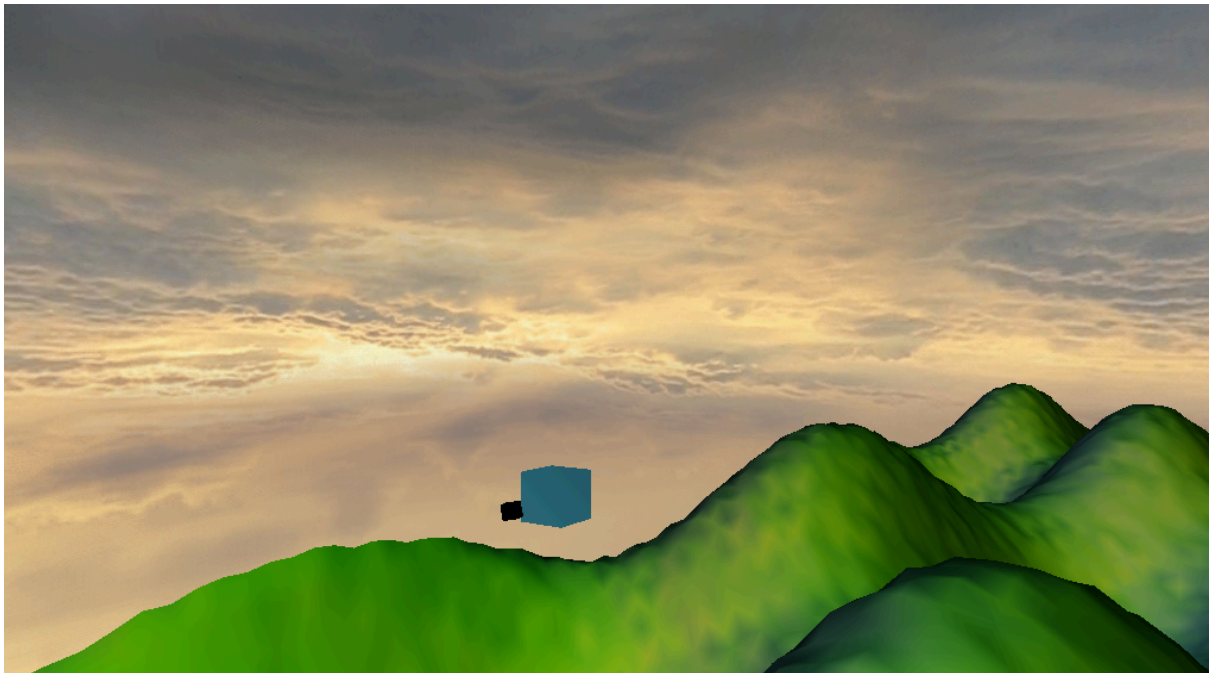

```
// ===== CUBEMAP =====  
GLuint Skybox::loadCubemap(const std::string& base_path)  
{  
    GLuint texID;  
    glGenTextures(1, &texID);  
    glBindTexture(GL_TEXTURE_CUBE_MAP, texID);  
  
    std::vector<std::string> faces =  
    {  
        base_path + "0.png", // +X  
        base_path + "1.png", // -X  
        base_path + "2.png", // +Y  
        base_path + "3.png", // -Y  
        base_path + "4.png", // +Z  
        base_path + "5.png"  // -Z  
    };  
  
    int width, height, channels;
```

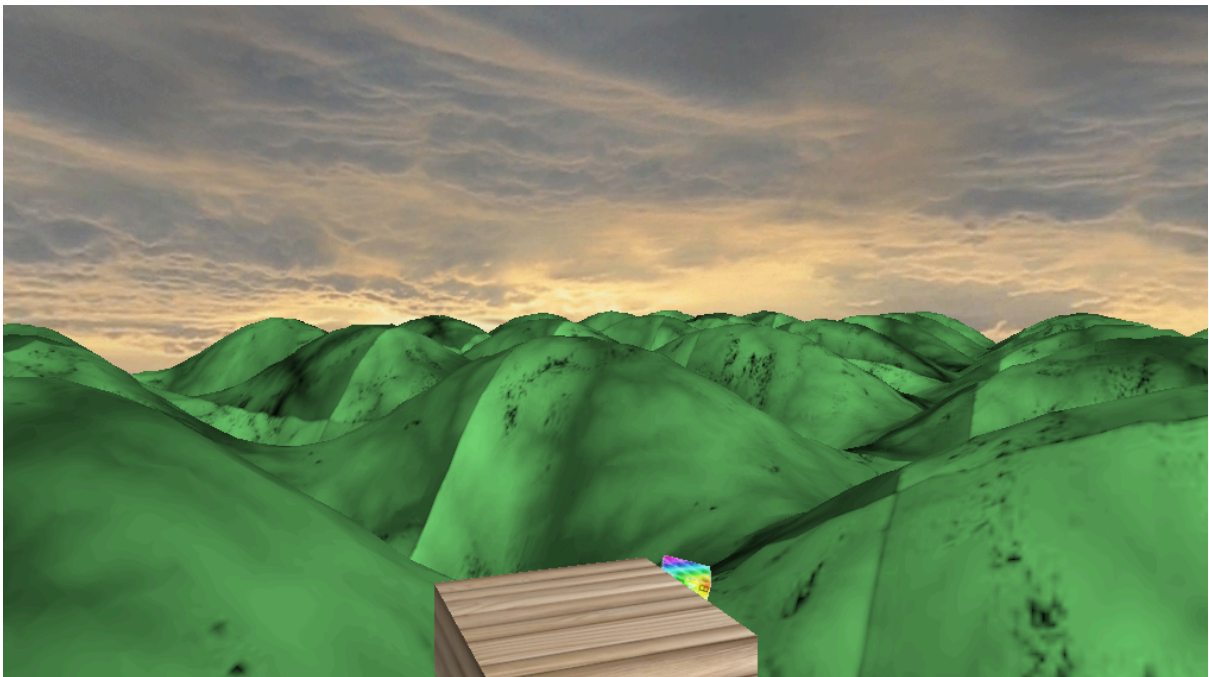
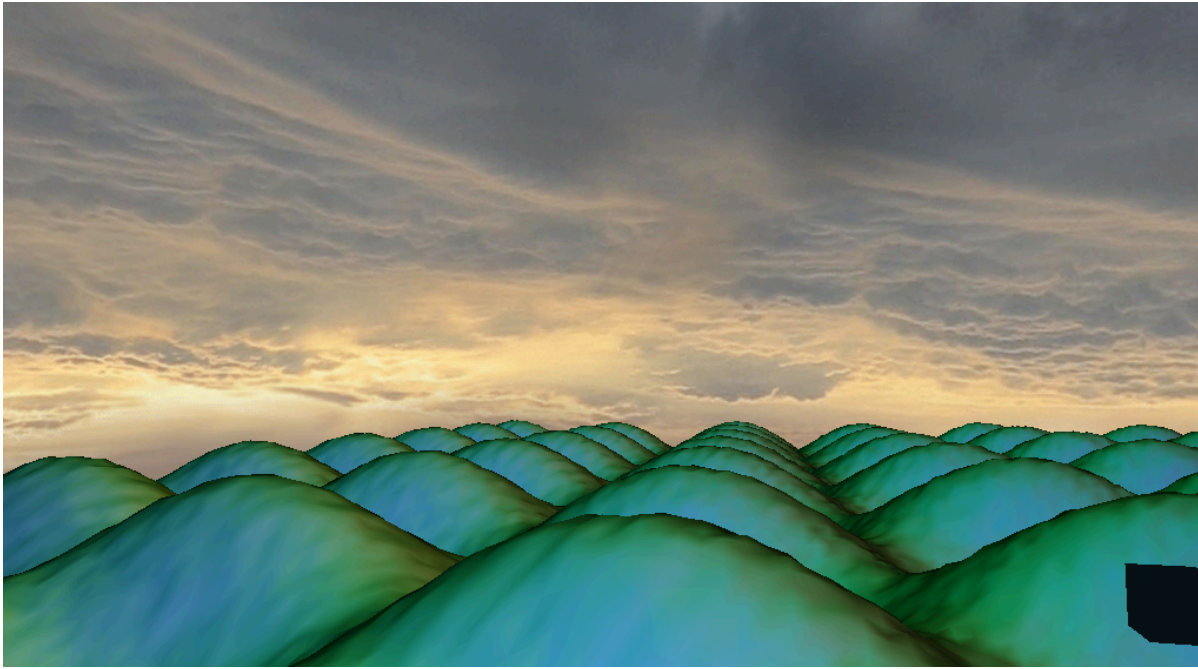
En este caso, he cargado y generado el Cubemap con las diferentes imágenes para poner como fondo y las he ido poniendo en cada una de las caras del cubo para llegar a forma el fondo o el skybox. En cada cara le corresponde una coordenada diferente.

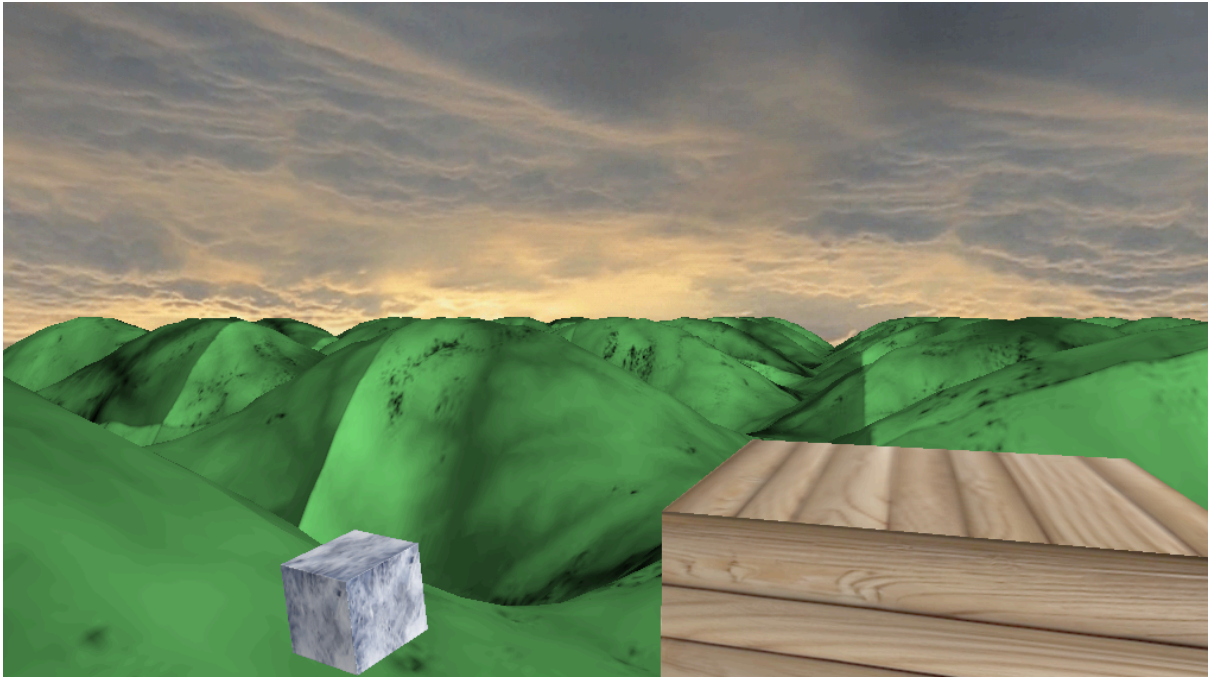
```
// ORDEN DE CARAS (IMPORTANTE)  
// =====  
static const GLenum targets[6] =  
{  
    GL_TEXTURE_CUBE_MAP_NEGATIVE_Z,  
    GL_TEXTURE_CUBE_MAP_NEGATIVE_X,  
    GL_TEXTURE_CUBE_MAP_POSITIVE_Z,  
    GL_TEXTURE_CUBE_MAP_POSITIVE_X,  
    GL_TEXTURE_CUBE_MAP_POSITIVE_Y,  
    GL_TEXTURE_CUBE_MAP_NEGATIVE_Y  
};
```

Serían estas de aquí, donde las hemos definido anteriormente y donde definimos el número de targets que son las caras (6).

El entorno del paisaje está encapsulado por un **Skybox** que simula un horizonte infinito. Técnicamente se compone de un mapa de cubo (**Cube Map**) formado por 6 texturas cuadradas que representan las caras de un cubo que envuelve la escena. Durante su renderizado, se deshabilita la escritura en el buffer de profundidad (`glDepthMask(GL_FALSE)`). El shader del Skybox iguala artificialmente su coordenada de profundidad Z al valor máximo (1.0), asegurando que el fondo siempre se dibuje por detrás de cualquier objeto físico del terreno, sin importar lo lejos que navegue la cámara.







Como se puede ver cada cara es diferente y cada una de las caras están en diferentes coordenadas tanto positivas como negativas para que no se mezclen unas con otras.

Optimización

Con el objetivo de maximizar el rendimiento global de la aplicación y garantizar una tasa de fotogramas estable, se han aplicado diversas técnicas de optimización propias de sistemas de renderizado en tiempo real basados en OpenGL. Estas optimizaciones se centran en reducir la carga de la GPU y minimizar las operaciones costosas en la CPU durante el bucle de renderizado.

Descarte de caras (Face Culling)

Se ha activado el mecanismo de *face culling* mediante `glEnable(GL_CULL_FACE)`, que permite descartar automáticamente las caras traseras de los modelos geométricos.

Dado que en la escena el usuario nunca visualiza el interior de los objetos cerrados ni la cara inferior del terreno, este proceso evita el cálculo innecesario de triángulos que no contribuyen al resultado final. Como consecuencia, se reduce significativamente la carga del rasterizador y se mejora el rendimiento general.

Minimización de cambios de estado

Otra de las optimizaciones aplicadas consiste en la reducción de cambios de estado dentro del pipeline de OpenGL.

El cambio de programa de sombreado (`glUseProgram`) es una operación costosa a nivel de CPU, por lo que se ha optado por agrupar los objetos según el shader y material que utilizan. De este modo, se realizan menos cambios de contexto durante el renderizado, lo que disminuye el overhead y mejora la eficiencia del sistema.

Optimización del uso de buffers (VBO y EBO)

La geometría de la escena se almacena en la memoria de la GPU mediante **Vertex Buffer Objects (VBO)** y **Element Buffer Objects (EBO)**.

Estos buffers se inicializan una única vez con la bandera `GL_STATIC_DRAW`, indicando que los datos no se modifican durante la ejecución. Esto permite que la GPU gestione directamente la geometría sin necesidad de retransmitirla continuamente desde la CPU.

El uso de índices mediante EBO evita la duplicación de vértices, reduciendo el consumo de memoria y mejorando la eficiencia del renderizado.

Reducción del tráfico CPU–GPU

Se ha minimizado la comunicación entre CPU y GPU durante el bucle de renderizado.

En lugar de enviar geometría completa en cada frame, únicamente se transmiten datos ligeros como matrices de transformación (model, view y projection). Esto reduce significativamente el ancho de banda utilizado y mejora el rendimiento en tiempo real.

Resumen de optimizaciones

En conjunto, las optimizaciones implementadas se pueden resumir en los siguientes puntos:

- Eliminación de geometría no visible mediante face culling
- Reducción de cambios de estado en el pipeline gráfico
- Uso eficiente de VBO y EBO con datos estáticos en GPU
- Minimización de transferencias CPU–GPU durante el renderizado

Conclusiones

El desarrollo de este proyecto ha permitido aplicar de forma práctica los principales conceptos de la programación gráfica moderna, integrando múltiples sistemas dentro de una arquitectura coherente basada en OpenGL.

A lo largo del proceso se han implementado elementos fundamentales como la gestión de shaders, el uso de buffers optimizados (VBO y EBO), la generación procedural de terrenos, el control de cámara en primera persona y la organización jerárquica de objetos mediante un grafo de escena. Asimismo, se ha logrado incorporar correctamente el uso de materiales y texturas, lo que ha contribuido significativamente a mejorar el realismo visual y la calidad final de la escena.

Uno de los principales resultados del proyecto es la construcción de una escena 3D funcional, interactiva y optimizada, capaz de representar múltiples objetos con iluminación, texturas y animaciones en tiempo real. La correcta integración de estos elementos demuestra una comprensión práctica del pipeline gráfico y de la gestión eficiente de recursos.

Además, la arquitectura modular empleada facilita la escalabilidad del sistema, permitiendo la incorporación de nuevas funcionalidades sin comprometer la estructura existente, lo cual es clave en el desarrollo de aplicaciones gráficas complejas.

Como posibles líneas de mejora futura, se podrían integrar técnicas más avanzadas como sombras dinámicas en tiempo real, iluminación global, efectos de postprocesado o sistemas de partículas, acercando el proyecto a las capacidades de un motor gráfico profesional.

En conclusión, el proyecto cumple los objetivos planteados y evidencia una comprensión sólida de los principios fundamentales del renderizado 3D moderno, así como la capacidad de aplicar dichos conocimientos en un entorno práctico y funcional.

